



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Árboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3B
Alumnos participantes:	Amaguaña Villacrés Shirley Dayana
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles.

Específicos:

- Comprender la estructura de nodos en árboles N-arios y árboles binarios.
- Implementar el algoritmo de inserción en un Árbol Binario de Búsqueda (BST).
- Calcular la altura máxima de un árbol binario utilizando recursividad.
- Realizar el recorrido InOrder de un árbol binario y obtener los valores ordenados.
- Transformar un árbol binario en su imagen espejo intercambiando los subárboles.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computador, Herramientas ofimáticas e Internet.
- Bibliografía de la asignatura
- Material disponible en el aula virtual de la asignatura

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar



Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de baja complejidad utilizando árboles.

Link de GitHub: https://github.com/shirley6126/Ape3_Arboles

Introducción

Los árboles son una de las estructuras de datos más importantes en la informática. A diferencia de las listas o los arreglos, los árboles organizan la información de forma jerárquica, lo que los hace muy útiles para representar relaciones entre elementos, como sistemas de archivos, bases de datos, expresiones matemáticas y muchos otros contextos.

En esta práctica se trabajó con dos tipos principales de árboles: los árboles N-arios, donde cada nodo puede tener cualquier número de hijos, y los árboles binarios, donde cada nodo tiene a lo sumo dos hijos (izquierdo y derecho). Dentro de los árboles binarios, se estudió el BST (Árbol Binario de Búsqueda), que mantiene un orden específico para facilitar las búsquedas.

Desarrollo

Ejercicio 1: Árboles N-arios – Conteo de Nodos

En este ejercicio se implementó la función contarNodos() para un árbol N-ario. Esta función recibe el nodo raíz y devuelve el total de nodos existentes en el árbol. El árbol de prueba contiene 6 nodos distribuidos en dos niveles.

Código implementado:

En C++



```
#include <iostream>
#include <vector>
using namespace std;

struct NodoN {
    int valor;
    vector<NodoN*> hijos;
    NodoN(int v) : valor(v) {}
};

int contarNodos(NodoN* raiz) {
    // Caso base: si el nodo es nulo no hay nada que contar
    if (raiz == nullptr) return 0;

    // Empezamos contando el nodo actual
    int total = 1;

    // Recorremos cada hijo y sumamos sus nodos
    for (NodoN* hijo : raiz->hijos) {
        total += contarNodos(hijo);
    }

    return total;
}

int main() {
    // Construimos el árbol
    NodoN* n5 = new NodoN(5);
```

En Java



```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

class NodoN {
    public int valor;
    public List<NodoN> hijos;

    public NodoN(int valor) {
        this.valor = valor;
        this.hijos = new ArrayList<>();
    }

    public NodoN(int valor, List<NodoN> hijos) {
        this.valor = valor;
        this.hijos = hijos;
    }
}

public class Ejercicio1_Basico {
    public static int contarNodos(NodoN raiz) {
        // Caso base: si el nodo es nulo retornamos 0
        if (raiz == null) return 0;

        // Contamos el nodo actual
        int total = 1;

        // Sumamos los nodos de cada hijo de forma recursiva
        for (NodoN hijo : raiz.hijos) {
            total += contarNodos(hijo);
        }

        return total;
    }
}

Run | Debug
public static void main(String[] args) {
```

Salida en consola:

```
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```

Explicación de la lógica:

Se utilizó un índice entero para recorrer el arreglo de hijos de cada nodo, lo que permite controlar manualmente la iteración. La función inicia en 1 para contar el nodo actual, luego itera por cada hijo llamándose recursivamente y acumulando el resultado. El caso base es un nodo nulo que retorna 0 inmediatamente.



Ejercicio 2: Árbol Binario – Inserción en BST

Se implementó insertar() para agregar nuevos valores en un Árbol Binario de Búsqueda. La función mantiene la propiedad del BST: valores menores al nodo actual se ubican a la izquierda, y valores mayores o iguales a la derecha.

Código implementado:

En C++

```
#include <iostream>
using namespace std;

struct Nodo {
    int valor;
    Nodo* izquierdo;
    Nodo* derecho;
    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
};

Nodo* insertar(Nodo* raiz, int valor) {
    // Si llegamos a un nodo nulo, creamos el nuevo nodo aquí
    if (raiz == nullptr) return new Nodo(valor);

    // Insertamos a la izquierda si el valor es menor
    if (valor < raiz->valor) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    }
    // Insertamos a la derecha si es mayor o igual
    else {
        raiz->derecho = insertar(raiz->derecho, valor);
    }

    return raiz;
}

int main() {
```

En Java

```
class Nodo {
    int valor;
    Nodo izquierdo, derecho;
    Nodo(int valor) { this.valor = valor; }
}

public class Ejercicio2_Binario {
    public static Nodo insertar(Nodo raiz, int valor) {
        // Si llegamos a null, creamos el nodo aquí
        if (raiz == null) return new Nodo(valor);

        // Menores van a la izquierda
        if (valor < raiz.valor) {
            raiz.izquierdo = insertar(raiz.izquierdo, valor);
        }
        // Mayores o iguales van a la derecha
        else {
            raiz.derecho = insertar(raiz.derecho, valor);
        }

        return raiz;
    }

    Run | Debug
    public static void main(String[] args) {
        Nodo raiz = new Nodo(valor: 10);
    }
}
```



Salida en consola:

```
--- Prueba Ejercicio 2 ---  
Raiz (Esperado 10): 10  
Hijo Izquierdo de Raiz (Esperado 5): 5  
Hijo Derecho de Raiz (Esperado 15): 15  
Hijo Izquierdo del 5 (Esperado 3): 3
```

Explicación de la lógica:

La función evalúa el valor a insertar comparándolo con el nodo actual. Si el lugar está vacío (nodo nulo), se crea el nuevo nodo ahí. De lo contrario, se decide el lado correcto y se llama recursivamente, actualizando el puntero del hijo con el nodo retornado para garantizar el correcto enlace de la estructura.

Ejercicio 3: Árbol Binario – Altura Máxima

Se implementó `calcularAltura()` para determinar la profundidad máxima de un árbol binario. El árbol de prueba es una cadena de tres nodos, por lo que la altura esperada es 3.

Código implementado:

En C++

```
#include <iostream>  
#include <algorithm> // util para std::max  
using namespace std;  
  
// Asumiendo la estructura Nodo del Ejercicio 2  
struct Nodo {  
    int valor;  
    Nodo* izquierdo;  
    Nodo* derecho;  
    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}  
};  
  
int calcularAltura(Nodo* raiz) {  
    // Si el nodo es nulo, la altura es 0  
    if (raiz == nullptr) return 0;  
  
    // Calculamos la altura de cada subárbol  
    int alturaIzq = calcularAltura(raiz->izquierdo);  
    int alturaDer = calcularAltura(raiz->derecho);  
  
    // La altura del árbol es el mayor entre los dos lados más 1 (por el nodo actual)  
    return 1 + max(alturaIzq, alturaDer);  
}
```

En Java



```
public class Ejercicio3_Binario2 {
    public static int calcularAltura(Nodo raiz) {
        // Árbol vacío tiene altura 0
        if (raiz == null) return 0;

        // Calculamos la altura de los dos lados
        int alturaIzq = calcularAltura(raiz.izquierdo);
        int alturaDer = calcularAltura(raiz.derecho);

        // Nos quedamos con el lado más largo y sumamos el nodo actual
        return 1 + Math.max(alturaIzq, alturaDer);
    }

    Run | Debug
    public static void main(String[] args) {
        //      1
        //      \
        //      2
        //      /
        //      3
        Nodo raiz = new Nodo(valor: 1);
        raiz.derecho = new Nodo(valor: 2);
        raiz.derecho.izquierdo = new Nodo(valor: 3);

        System.out.println(x: "--- Prueba Ejercicio 3 ---");
    }
}
```

Salida en consola:

```
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
```

Explicación de la lógica:

La función almacena en variables separadas (profIzq y profDer) la altura de cada subárbol, calculadas recursivamente. Luego retorna el mayor valor entre ambas más 1, que representa el nivel del nodo actual. Para el árbol nulo, el retorno directo es 0 sin realizar ninguna llamada adicional.

Ejercicio 4: Recorridos – InOrder

Se implementó el recorrido InOrder sobre un árbol BST completo de 7 nodos. Al aplicar este recorrido sobre un BST, los valores deben aparecer en orden ascendente: [1, 2, 3, 4, 5, 6, 7].

Código implementado:

En C++



```
#include <iostream>
#include <vector>
using namespace std;

struct Nodo {
    int valor;
    Nodo* izquierdo;
    Nodo* derecho;
    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
};

void inOrderAux(Nodo* nodo, vector<int>& resultado) {
    // Si el nodo es nulo, no hacemos nada
    if (nodo == nullptr) return;

    // Primero vamos al lado izquierdo
    inOrderAux(nodo->izquierdo, resultado);

    // Luego guardamos el valor del nodo actual
    resultado.push_back(nodo->valor);

    // Después vamos al lado derecho
    inOrderAux(nodo->derecho, resultado);
}

vector<int> recorridoInOrder(Nodo* raiz) {
    vector<int> resultado;
    inOrderAux(raiz, resultado);
    return resultado;
}

int main() {
```

En Java

```
import java.util.ArrayList;
import java.util.List;

public class RecorridoInOrder {
    public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
        // Si el nodo es null paramos la recursión
        if (nodo == null) return;

        // Primero recorremos el lado izquierdo
        inOrderAux(nodo.izquierdo, resultado);

        // Agregamos el valor del nodo actual
        resultado.add(nodo.valor);

        // Después recorremos el lado derecho
        inOrderAux(nodo.derecho, resultado);
    }

    public static List<Integer> recorridoInOrder(Nodo raiz) {
        List<Integer> resultado = new ArrayList<>();
        inOrderAux(raiz, resultado);
        return resultado;
    }
}

Run | Debug
public static void main(String[] args) {
    //      4
    //    /  \
    //   2    6
    //  / \  / \
    // 1  3 5  7
```



Salida en consola:

```
--- Prueba Ejercicio 4 ---  
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]  
Tu resultado:      [1, 2, 3, 4, 5, 6, 7]
```

Explicación de la lógica:

La función auxiliar `inOrderAux()` recibe el nodo actual y la lista de resultados. El orden de visita es: subárbol izquierdo, nodo actual, subárbol derecho. Se detiene cuando el nodo es nulo. Como el árbol de prueba es un BST balanceado, el resultado del recorrido coincide con la secuencia ordenada de los valores.

Ejercicio 5: Transformación – Árbol Espejo

Se implementó `invertir()` para transformar el árbol en su imagen especular. La función intercambia los hijos izquierdo y derecho de cada nodo de manera recursiva. El árbol de prueba tiene raíz 1, hijo izquierdo 2 e hijo derecho 3.

Código implementado:

En C++

```
#include <iostream>  
using namespace std;  
  
struct Nodo {  
    int valor;  
    Nodo* izquierdo;  
    Nodo* derecho;  
    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}  
};  
  
Nodo* invertir(Nodo* raiz) {  
    // Si el nodo es nulo no hay nada que invertir  
    if (raiz == nullptr) return nullptr;  
  
    // Invertimos los subárboles de forma recursiva  
    Nodo* izqInvertido = invertir(raiz->izquierdo);  
    Nodo* derInvertido = invertir(raiz->derecho);  
  
    // Intercambiamos los hijos para hacer el espejo  
    raiz->izquierdo = derInvertido;  
    raiz->derecho = izqInvertido;  
  
    return raiz;  
}  
  
int main() {
```

En Java

```
public class Ejercicio5_Transformacion {  
    public static Nodo invertir(Nodo raiz) {  
        // Si el nodo es nulo no hay nada que hacer  
        if (raiz == null) return null;  
  
        // Invertimos cada subárbol primero  
        Nodo izqInvertido = invertir(raiz.izquierdo);  
        Nodo derInvertido = invertir(raiz.derecho);  
  
        // Intercambiamos los hijos para obtener el espejo  
        raiz.izquierdo = derInvertido;  
        raiz.derecho = izqInvertido;  
  
        return raiz;  
    }  
  
    Run | Debug  
    public static void main(String[] args) {
```



Salida en consola:

```
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```

Explicación de la lógica:

Se utilizó una variable temporal para realizar el intercambio de punteros sin perder referencias. Primero se intercambian los hijos del nodo actual y luego se llama recursivamente a cada subárbol (ya en su nueva posición). Este enfoque asegura que todo el árbol quede correctamente invertido de arriba hacia abajo.

Diferencias entre C++ y Java en los ejercicios

La diferencia principal es la gestión de memoria: en C++ los nodos se crean con `new` y el programador es responsable de liberarlos con `delete`, mientras que en Java el recolector de basura lo hace automáticamente. Además, C++ usa punteros (`Nodo*`) y accede a los miembros con `->`, mientras que Java usa referencias de objeto y accede con `..`. En cuanto a la estructura, C++ define los nodos con `struct` y las funciones son independientes, mientras que Java los define con `class` y los métodos pertenecen a la clase.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Las estructuras dinámicas de árboles constituyen una alternativa al procesamiento secuencial de los datos, de amplia aplicación práctica.

- El BST organiza automáticamente los datos durante la inserción, lo que simplifica enormemente las búsquedas posteriores sin necesidad de ordenar manualmente los elementos.
- El recorrido InOrder demuestra una propiedad muy útil del BST: al visitarlo con este patrón se obtienen los datos en orden ascendente, lo que puede usarse como algoritmo de ordenamiento implícito.
- La transformación espejo evidenció que modificar la estructura de un árbol es posible simplemente reordenando los punteros, sin necesidad de crear nuevos nodos ni estructuras adicionales.

2.10 Recomendaciones

Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar

- Dibujar el árbol en papel antes de programar cualquier algoritmo ayuda mucho a visualizar el comportamiento esperado y detectar posibles errores lógicos antes de escribir código.



- Es importante definir siempre el caso base de la recursión antes de escribir el caso recursivo, para evitar bucles infinitos o errores en tiempo de ejecución.

2.11 Referencias bibliográficas

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed. Cambridge, MA, EE.UU.: MIT Press, 2022.

[2] B. Stroustrup, *The C++ Programming Language*, 4.^a ed. Upper Saddle River, NJ, EE.UU.: Addison-Wesley, 2013.

[3] J. Bloch, *Effective Java*, 3.^a ed. Boston, MA, EE.UU.: Addison-Wesley, 2018.

2.12 Anexos